Invoice Intake — Testing Guide

Step-by-step walkthrough of every flow, with what to expect at each point and why it behaves that way.

> **The one idea behind the whole system:** the model *transcribes*, our code *computes*. The AI is only ever asked what characters are printed on the page. Every date conversion, yen amount, tax code and supplier lookup is deterministic Python with tests — because those are exactly where a language model fails quietly, and a quiet failure in accounts payable is a wrong payment.

0 · Start it — one command

From the project folder:

```
# the .env with your OpenRouter key is already in place
docker compose up --build
```

Leave that terminal running; it streams the logs. First run builds the images (1–2 min), then the API is reachable in about 15 seconds. **Nothing is pre-loaded** — the app opens on an empty upload screen, which is the point.

In a second terminal, confirm all four services are up:

```
docker compose ps
```

SERVICE	URL	WHAT IT IS
web	`http://localhost:3000`	The app — start here
api	`http://localhost:8001/docs`	Pipeline API (interactive OpenAPI docs)
accounting	`http://localhost:8080`	The client's system, run **verbatim** from the brief
db	`localhost:5433`	PostgreSQL — audit trail and review queue

> **Why 8001 and 5433?** Your machine already has something on 8000 and a native PostgreSQL on 5432. Rather than ask you to stop them, the container publishes on the next port up. Inside the compose network the services still talk on 8000 and 5432.

1 · Upload one invoice — the main flow

1. Open `http://localhost:3000`. You get an empty screen with a drop zone.
2. Drag `invoices/invoice_01.pdf` onto it (or click to browse).

WHAT SHOULD HAPPEN — WATCH IT, DO NOT RELOAD

- The invoice appears **immediately** under *Being read* with a pulsing `reading…` pill.
- After ~10 seconds the row moves itself to **Registered**, filled in: supplier `P-1001`, invoice `YM-2026-0107`, ¥334,400, accounting ID `ACC-0001`.
- No page reload. The screen polls while anything is being read, and stops when nothing is.

That is the product. Someone has an invoice in front of them, drops it in, and it is read, checked and filed. The client's staff handle invoices "one by one, as they arrive from suppliers" — so that is the shape the intake takes. Everything below is what happens when an invoice *cannot* just be filed.

Now upload the same file again

Drag `invoice_01.pdf` in a second time.

EXPECTED

It is refused before anything is read: "*this exact file has already been uploaded*", with a link to the one already here. Documents are keyed by SHA-256 of their bytes, so re-uploading costs nothing and registers nothing.

Try something that is not an invoice

Drop `README.md` on it.

EXPECTED

"*.md is not supported. Upload a PDF, JPG, PNG, TIFF or WebP.*" Rejected on the spot, with no half-created record left behind.

2 · Load the rest — the bulk path

Uploading one at a time is the daily flow. For a backlog — or to see all twelve cases at once — use the bulk path.

1 Click **process the 12 sample invoices** under the drop zone (visible when the list is empty), or run:

```
docker compose exec api python -m app.cli ingest
```

Takes ~2.5 minutes. Rows appear as each one finishes.

WHAT YOU SHOULD END UP WITH

COUNT	GROUP	MEANING
7	REGISTERED	Passed every check. No human involved.
3	NEEDS REVIEW	A check failed, or policy requires a person.
2	BLOCKED	Cannot be registered at all.

10 of 12 will end up registered. Not 12 — and that is the correct answer.

Notice the ordering: **blocked first, registered last**. The queue is sorted by what a human has to decide, not by arrival time.

Read the stat cards

- **Auto-pass rate** — the only metric that decides whether this pays for itself. The AI costs less than half a cent per invoice; *review minutes* are the real cost.
- **Extraction cost** — measured live from OpenRouter's billing, not estimated.
- **In accounting** — read back live from `GET /invoices` on the client's system.

> **Don't read 58% as a production number.** Five of these twelve invoices are deliberately broken. A real month is mostly repeat suppliers with stable layouts.

3 · Flow A — the duplicate BLOCKED

This is the failure the CEO's email describes, planted in the sample data.

1. Click `invoice_07.jpg` in the Blocked group.

WHAT TO LOOK FOR

- Banner: **BLOCKER · dedupe.exact** — "Already registered: invoice YM-2026-0107 for this supplier was posted as ACC-0001 from **invoice_01.pdf**"
- Scroll to **All checks** — **every single one passes.** The reading is perfect.
- The **Approve & register** button is greyed out.

Why this matters. invoice_07 is a scan of invoice_01 — same supplier, same invoice number, same ¥334,400. Extraction accuracy would never have caught it, because nothing was misread. It is caught by checking our own records *before* calling the accounting system, which means the reviewer is told *which* invoice it duplicates rather than just seeing a failure.

2. Try to force it through — confirm a blocker is not an opinion:

```
curl -s -X POST localhost:8001/api/invoices/7/approve \
  -H 'Content-Type: application/json' -d '{"actor":"you","note":"force it"}' | python3 -m json.t
ool
```

EXPECTED

`HTTP 409` naming the conflicting record. The UI button being disabled is a courtesy; the *API* is the thing that refuses.

4 · Flow B — the unknown supplier BLOCKED

1 Open `invoice_10.jpg`.

WHAT TO LOOK FOR

- **BLOCKER · partner.resolved** — 新星ロジスティクス株式会社 is not in the partner master.
- Supplier dropdown reads **— not resolved —**. There is no correct value to pick.

Why it is blocked rather than guessed. Only suppliers in `GET /partners` can be registered, so there is literally no `partner_code` to post against. The system will never auto-create a payee record — creating one is how invoice fraud succeeds. Someone with authority must add the supplier deliberately.

At scale this is the #1 queue filler — not OCR errors. Every new supplier, renamed entity and subsidiary stops here. It is the top item in "what I'd do with another 8 hours".

5 · Flow C — handwriting that matters vs. handwriting that doesn't

The sharpest judgement call in the build. Two invoices carry pen; they get different outcomes on purpose.

1 Open `invoice_04.jpg` — it is in the **Registered** group.

EXPECTED

It auto-posted, but `handwriting.detected` shows as a **WARN**: a 受領 ("received") stamp. Workflow noise. It cost nobody any time.

2 Open `invoice_08.jpg` in **Needs review**. Look at the document image — find the **red pen on the bank account line** (「…に変更」).

EXPECTED — TWO HANDWRITING CHECKS, DIFFERENT SEVERITIES

- **ERROR · handwriting.on_payment_details** — must be confirmed with the supplier
- **WARN · handwriting.detected** — also picked up the 至急 ("urgent") stamp separately

> **The point:** bank details are *not part of the accounting API's payload at all*. If this pipeline did not flag it, **nothing downstream ever would**. A changed payee account is the classic invoice-fraud vector. A system that treated "has handwriting" as one signal would either block everything or miss this.

6 · Flow D — the supplier's own invoice is wrong REVIEW

The best case in the set: the extraction is perfect and the document is defective.

1 Open `invoice_09.pdf`.

WHAT TO LOOK FOR — THE LIVE RECALCULATION BLOCK

LINE	VALUE
Subtotal from lines	¥134,088
Tax 10% on ¥134,088 (rounded down)	¥13,408
Total the accounting system will store	**¥147,496**
Total printed on the document	**¥147,497**

What happened. The supplier floored the tax on the tax line (13,408.8 → 13,408) but rounded it *up* when computing the total. Their invoice is internally inconsistent by ¥1. This is a defect in the document, not in the reading of it — the model transcribed 147,497 faithfully, exactly as instructed, instead of "helpfully" correcting it and destroying the evidence.

Why this is the strongest argument for the design. A human keying this in would have typed 147,497 straight through. The accounting system recalculates from the line items and would have rejected it. The check catches a *supplier* error, not just an AI error.

2 Type a note in the **Note** field, then click **Approve & register**.

EXPECTED

Status flips to POSTED with an accounting ID. It registers **¥147,496** — the recalculated figure, not the printed one.

3 Confirm what actually landed in the client's system:

```
curl -s -H 'X-API-Key: demo-key-1234' localhost:8080/invoices \
  | python3 -c "import json,sys; [print(r) for r in json.load(sys.stdin)['data']['invoices'] if r['invoice_number']=='OSK-26-0128']"
```

EXPECTED

`"total_amount": 147496`

Try editing before you approve

Change any **Amount** in the line items table and watch the recalculation update as you type. This is the accounting system's *own* arithmetic — tax per code, rounded down — reproduced in the browser, so a correction that the API would reject is visible *before* you save it.

7 · Flow E — a policy stop, not a doubt REVIEW

1 Open `invoice_02.pdf`.

EXPECTED

- Only one failed check: **ERROR · amount.threshold** — ¥1,560,988 exceeds the ¥1,000,000 auto-approval limit.
- Everything else passes. Scroll the document — **26 line items across 2 pages**, with the totals printed only on page 2, all read correctly.

This one is not stopped because the machine is unsure. It is stopped because a finance team would demand a human on large amounts regardless of confidence. That is a **policy dial**, not a correctness check — set via `AMOUNT_REVIEW_THRESHOLD_JPY` in `.env`.

So the three review items are three *different kinds* of reason: a document defect (09), a fraud-shaped annotation (08), and a policy control (02).

8 · The audit trail — how you'd find a bad registration

After approving, check who accepted responsibility for what:

```
docker compose exec -T db psql -U invoice -d invoice -c \
  "select i.invoice_number, r.actor, r.action, r.note from review_events r
     join invoices i on i.id=r.invoice_id order by r.id;"
```

EXPECTED

Each approval records the actor, the note, **and which checks they overrode** — e.g. `overrode 1 check(s): arithmetic.total`. "Registered because a person overrode the totals check" is a query, not an investigation.

Three tables exist purely for this: `check_results` (every verification decision), `postings` (the exact bytes sent and received), `review_events` (who changed what, and what it looked like before). Any figure in the ledger traces back to the pixels it came from.

9 · Idempotency — run it twice

The brief invites you to retry as often as you like. A pipeline that double-registers on a second run would reproduce the client's original complaint.

```
docker compose exec api python -m app.cli ingest
```

EXPECTED

`processed 0 new document(s)` — every file is skipped. Documents are keyed by SHA-256 of their bytes, so re-ingesting is a no-op. Nothing is registered twice.

10 · The test suite — verification without the AI

```
cd backend && .venv/bin/python -m pytest -q
```

EXPECTED

86 passed in ~0.2s. No LLM, no database, no network.

The important one is `tests/test_ground_truth.py`: it pushes the *correct* transcription of all 12 invoices (hand-transcribed in `evals/ground_truth.yaml`) through the real check ladder and asserts each one routes to the right place. When it fails, the business logic is wrong — not the model having a bad day.

11 · The model comparison

```
# costs a few cents; results already recorded in evals/results/
make eval
```

MODEL	FIELD ACC.	PERFECT	$/INVOICE	VERDICT
gemini-3.7-flash	100%	12/12	$0.0037	**chosen**
claude-sonnet-5	100%	12/12	$0.0200	5.4× the price, identical result
gemini-2.5-flash-lite	98.7%	10/12	$0.0005	7× cheaper, one real error
gemma-4-31b-it:free	0%	0/12	$0.0000	HTTP 429 on 11 of 12 calls

> **The finding worth reading.** On invoice_08 the cheap model read the tax total as **9,036** where the page prints **8,936** — while its *own* tax breakdown rows (680 + 8,256) sum to 8,936. It contradicted itself by one digit. `arithmetic.tax_per_code` caught it instantly, with no second model call, because the invoice prints its own tax breakdown and the document therefore contains everything needed to detect the error. That is what makes a 7×-cheaper model a real option rather than a gamble.

12 · Start over

```
# clear everything and go back to the empty upload screen
curl -X POST localhost:8001/api/admin/reset

# or a full clean rebuild from nothing
docker compose down -v && docker compose up --build
```

Either way you land back on an empty upload screen, ready to drop an invoice on.

Quick reference — all 12 invoices

FILE	OUTCOME	THE TRAP IT TESTS
invoice_01.pdf	AUTO	baseline, PDF with text layer
invoice_02.pdf	REVIEW	26 line items over 2 pages; over the amount limit
invoice_03.pdf	AUTO	mixed 8%/10% tax, floor rounding 6,067.2→6,067
invoice_04.jpg	AUTO	handwritten 受領 stamp (harmless); slash dates
invoice_05.jpg	AUTO	式 rows with null quantity and unit price
invoice_06.jpg	AUTO	supplier printed as a katakana *alias*
invoice_07.jpg	BLOCKED	**duplicate of 01** — the CEO's email
invoice_08.jpg	REVIEW	**red pen on the bank account**
invoice_09.pdf	REVIEW	no text layer; **printed total off by ¥1**
invoice_10.jpg	BLOCKED	**supplier not in the master**
invoice_11.jpg	AUTO	Japanese era dates 令和8年 → 2026
invoice_12.jpg	AUTO	negative line 値引き printed as △30,000

Invoice Intake — take-home submission by Sifat Sikder. Full reasoning in `SUBMISSION.md`; how to run it in `README.md`; 3-minute demo script in `docs/DEMO.md`.